WEB-07 | FINAL PROJECT REPORT

TransitNotice

Public Transport Service Update Tracker for Tamil Nadu

Field	Details
Team	Team 01
Student name	DIVYESH HARI G
Academic year	2026 - 27
Programme	Cybernaut EdTech - Web Scraping (SRS 2026), project WEB-07
Report date	02 November 2026

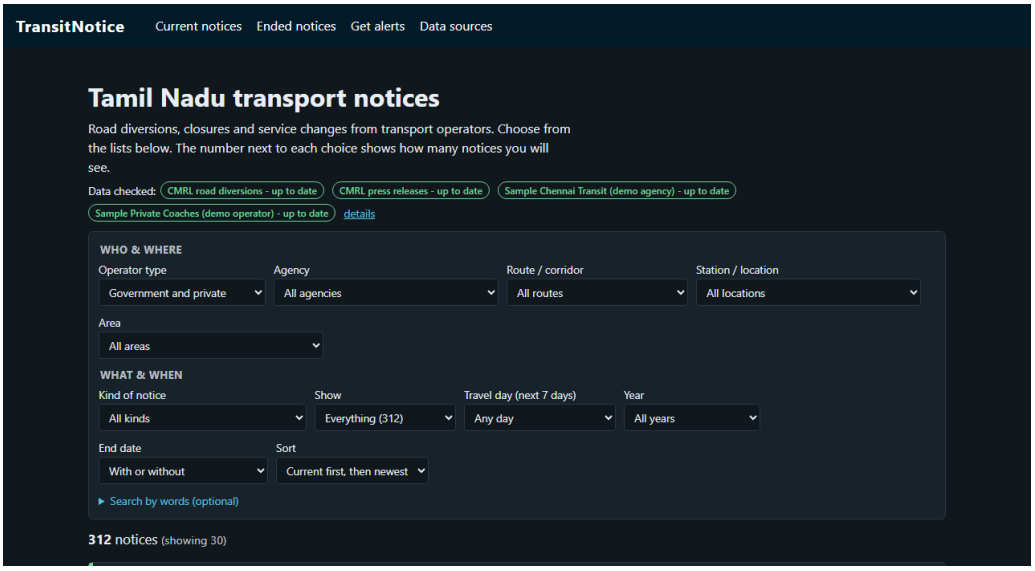
Abstract

Passengers in Chennai lose time when they do not hear about road diversions, station-entry closures, revised timings or special services. Official notices exist, but they are spread across web pages, rarely state an end date and are silently edited or removed. **TransitNotice** collects official, publicly available notices on a schedule, extracts the affected routes, stations, area, type and validity window, keeps every revision, and publishes a searchable website where each notice shows **when it was last verified and which official page it came from**. A failing source is shown as stale or unreachable - never as live.

The live system tracks two official Chennai Metro Rail Limited (CMRL) sources - Phase 2 road diversions and passenger-service press releases (2011-2026) - plus a demo government agency and a demo private operator that demonstrate the RSS path for private operators. Every filter is a drop-down list whose options are computed from the data, so every option a user can pick returns results.

312	4	129	59	0 / 595
notices tracked	sources (2 real CMRL)	filter options in 10 filters	automated tests pass	empty results in live filter check

Technology: Python 3.13, Flask, SQLite, standard-library urllib / html.parser / xml.etree / zoneinfo. No paid APIs, no login, no CAPTCHA handling.



The TransitNotice website: freshness of every source, drop-down filters with counts.

1. Context, stakeholders and success criteria

Problem. Metro construction in Chennai (Phase 2, corridors 3, 4 and 5) has closed or diverted dozens of roads, and CMRL regularly changes entrances, parking, timings and frequencies. The information is published, but a passenger has to know where to look, cannot filter it, and cannot tell whether a notice is still valid. The project goal is a monitoring application for official public notices that makes **freshness** and **origin** visible.

User personas

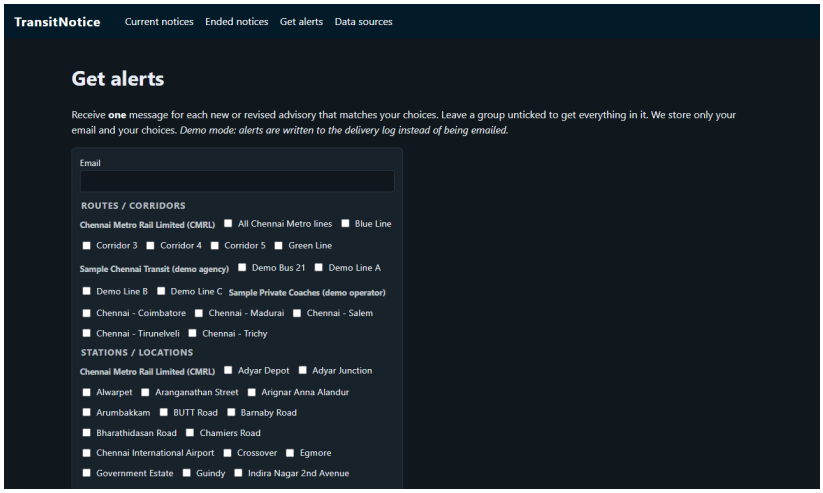
Persona	Needs	How TransitNotice helps
Priya, daily commuter (Porur - Poonamallee, Corridor 4)	Know before leaving home whether her road or line is affected today	Choose Corridor 4 and Today from drop-downs; one alert per new or revised notice
Ravi, MTC bus / auto driver	Which junctions are diverted and the alternative route	Location drop-down (e.g. Kodambakkam) with the official diversion text and link to the CMRL map page
Meera, help-desk / traffic staff	Answer 'is this still on, since when?' with a source	Start date, 'last verified' time, revision history and official notice link on every card

Success criteria (measurable)

#	Criterion	Target	Result (live, 02 Nov 2026)
S1	Notices parsed from live sources without error	≥ 95 %	43 / 43 road diversions; 261 / 262 passenger press releases (99.6 %)
S2	Expired notice never shown as active	0 violations	0 - 261 ended notices labelled 'Ended' (acceptance test)
S3	Duplicate alerts per revision per subscriber	0	0 - UNIQUE key + test
S4	Source failure never turns old data into live data	100 %	Verified by test and by a real CMRL timeout on the first run
S5	Every filter option returns at least one notice	100 %	595 live filter choices checked - 0 empty
S6	Source change visible on the site	≤ 1 refresh interval	5 min (demo sources), 6 h (CMRL - gentle on a public site)

Usefulness and limitations of scheduled public-source monitoring

Useful for	Limitations
One place to see all notices with filters; history of changes that agencies do not keep; honest freshness labels; alerts without a separate app per agency.	Not real time (up to one refresh interval old); only as good as the source (CMRL gives no end dates for diversions); withdrawal is inferred when a notice disappears; HTML layout changes break adapters (detected and flagged); terms of use must be re-checked; not an official channel.



Alert sign-up for Priya's use case: only the email is typed; routes, stations, areas and kinds are tick boxes.

2. Source selection, permissions and crawl policy

Source (key)	URL / method	Permission check	Volume
CMRL road diversions (cmrl) - Government	chennaimetrorail.org/wp-json/wp/v2/road-diversion (public WordPress list) + each notice page	robots.txt: User-agent: * / Disallow: (empty = allow all); pages are index, follow	43 notices; ~45 requests on first run, then 2 per refresh (detail pages cached until modified)
CMRL press releases (cmrl_press) - Government	chennaimetrorail.org/press-release/ (one HTML table, all years) + station list API	Same site, same robots.txt; PDFs are linked, not downloaded	1,334 releases read, 261 passenger notices kept; 2 requests per refresh
Sample Chennai Transit (sample_metro) - demo	HTML page served by the app (/demo-source/)	Own robots.txt allows /demo-source/	4 notices; mentors can change it live
Sample Private Coaches (demo_private_bus) - Private	RSS feed served by the app	Fictional operator, labelled 'demo' everywhere	4 notices; proves the private-operator path
MBTA (mbta) - optional	api-v3.mbta.com/alerts (open developer API)	robots 404; anonymous use allowed	Disabled by default

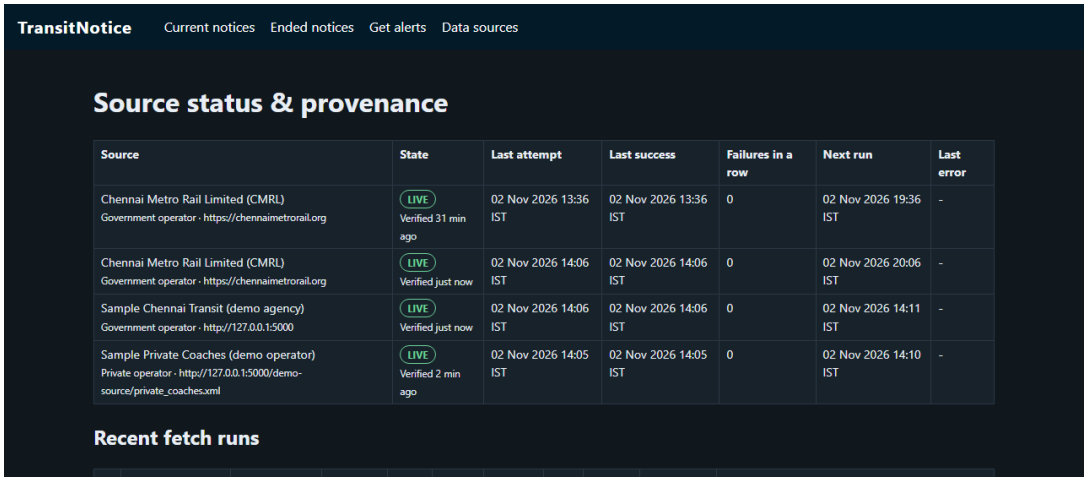
Other Tamil Nadu sources checked and not used

Source	Finding	Decision
Southern Railway (sr.indianrailways.gov.in)	robots.txt returns an error page - policy unreadable	Not crawled (unreadable robots = deny)
MTC Chennai, Chennai Port, AAI	TLS certificate chain cannot be verified	Not crawled (no TLS bypass)
PIB regional RSS, IntrCity	HTTP 403 to bots	Not crawled
TNSTC, Parveen, Namma Yatri, KPN, YoloBus	Booking apps; no notice page or feed	Nothing to collect
Freshbus, Zingbus	robots.txt blocks several crawlers; no feed	Not crawled
Chennai Traffic Police, SETC, SRS Travels	Domain does not resolve	Not reachable

No private Tamil Nadu operator currently publishes notices in a collectable form. The system is ready for them: an operator with an RSS/Atom feed is added in **agencies.json** with operator_type: Private, without code changes. Notices of real companies are never invented.

Crawl policy

- Honest User-Agent: TransitNoticeBot/1.0 (student project; contact: ...).
- robots.txt checked before each new site: 404 = allowed; error, 5xx, 401, 403 or unreadable = denied.
- Rate limit 1 request/second per host; timeout 15 s (30 s for the slow CMRL site); at most 3 attempts per request with 2 s and 4 s backoff; retries only for network errors, 429 and 5xx.
- After a failed refresh the next run is pushed back 10 min, 20 min, 40 min ... up to 6 h.
- No login, no CAPTCHA bypass, no private data. The only personal data is a subscriber's email and filter, given with consent; secrets live in .env, never in source code.



Data sources page: operator type, state, last attempt, last success, failures in a row and next scheduled run for every source (the fetch-run log follows below it).

3. Software requirements specification

Use cases

ID	Actor	Use case
UC1	Passenger	Browse notices by choosing operator type, agency, route, station, area, kind, status, travel day, year and end date from drop-downs; each option shows a count
UC2	Passenger	Open a notice: window, official link, last verification, revision history
UC3	Passenger	Subscribe (email + tick boxes); receive one alert per new or revised match
UC4	Passenger	Browse ended, withdrawn and cancelled notices (archive)
UC5	Scheduler	Refresh each source when due; back off after failures
UC6	Admin	Manual refresh, source health, fetch-run log, alert delivery log (token protected)
UC7	Integrator	Query /api/notices (JSON) with the same filters
UC8	Admin	Add a government or private operator via agencies.json (no code)

Requirements traceability

Requirement (brief sections 5 and 7)	Implementation
Collect public alerts from permitted sources	sources/cmrl.py, cmrl_press.py, feed.py; robots check in base.py
Extract route, type, issue and valid window	Adapters produce ParsedNotice; normalize.py (8 types); timeutil.py
Filter by route, station, area and service date	queries.search + notice_routes / notice_stations; faceted drop-downs
Active status from announced window	ingest.effective_status and queries.STATUS_SQL (computed at request time)
Track withdrawn, revised, extended	content hash, notice_revisions, describe_change, withdraw_missing
Last checked + provenance	last_confirmed_at, source_url, fetch_runs, Data sources page
Single alert per new / revised advisory	alert_log UNIQUE(notice_id, revision_id, subscriber_id)
Degrade gracefully	failed run never touches notices; per-item skip; ParseError guard
Backoff, timeout, retry ceiling, failure logs	Fetcher.get, refresh_source (next_run_at), fetch_runs
Constraints, idempotent upserts, access control, secrets	CHECK / UNIQUE; hash compare; admin token from env
Responsive, accessible UI and charts	CSS grid + mobile breakpoint; SVG charts with titles and data tables

Architecture

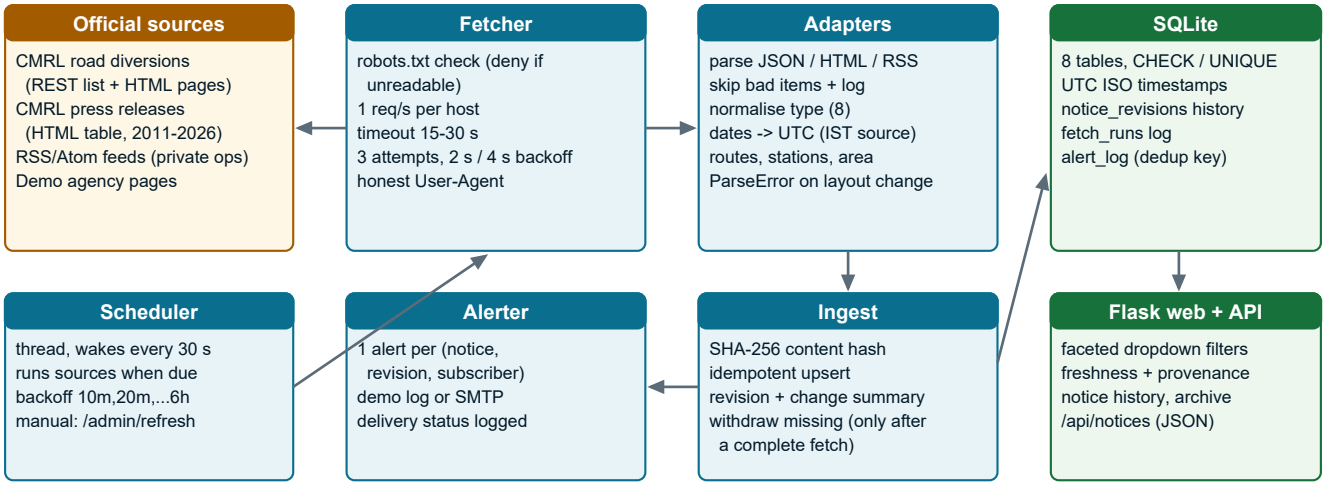


Figure 1 - Data flow from official sources to the website and alerts.

3.1 Database design (ER diagram)

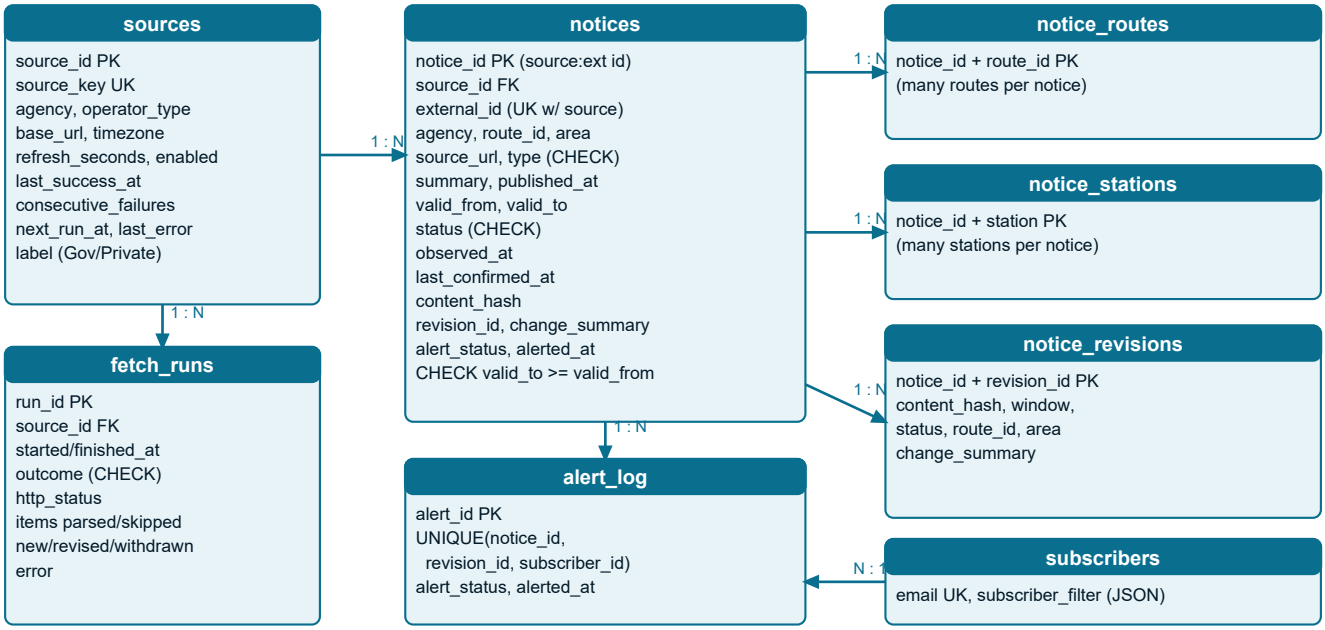


Figure 2 - SQLite schema (migrations/001_init.sql, 002_operator_type.sql).

Keys and repeat-run behaviour

- **notice_id = source_key:external_id** is stable (CMRL post id, press-release PDF name, RSS guid); UNIQUE(source_id, external_id) prevents duplicates.
- **Idempotent upsert:** a re-run with identical content only updates last_confirmed_at. Verified: three consecutive refreshes produce 0 new and 0 revised rows.
- **Revisions:** a different SHA-256 content hash (type, text, window, routes, stations, area, source status) increments revision_id, snapshots the row into notice_revisions and stores a readable change summary, e.g. Extended: end 02 Nov 2026 23:03 IST -> 03 Nov 2026 23:03 IST.
- **Withdrawal:** only after a complete, successful fetch are notices that the source no longer lists marked withdrawn (kept in the archive); items that could not be parsed are protected.
- **Constraints:** CHECK on type, status and outcome values; CHECK valid_to ≥ valid_from; foreign keys with cascade; schema_migrations table makes migrations run exactly once.

Time-zone policy

Rule	Example
Store UTC only (YYYY-MM-DDTHH:MM:SSZ) so text order = time order	2026-11-05T16:30:00Z
Naive source times use the source zone (Asia/Kolkata)	'2026-11-05 22:00' IST -> 16:30 UTC
Date-only end dates run to 23:59:59 local	'5 Nov 2026' -> 2026-11-05T18:29:59Z
'TBD', empty, 'until further notice'	valid_to = NULL (open ended)
Travel-day filter uses IST calendar days	'3 Nov' = 2 Nov 18:30 UTC to 3 Nov 18:29:59 UTC

4. Implementation

Collection. A Fetcher shared by all adapters checks robots.txt once per site, throttles to one request per second, and retries with exponential backoff. CMRL diversions are listed through the site's public WordPress REST endpoint; each detail page is parsed with an html.parser state machine (title, description paragraphs or list items, 'Diversion Start Date' and 'CMRL Package' cards, 'Last Updated On'). Detail pages are cached and re-downloaded only when the list reports a new modified time. Press releases come from one HTML table; titles are classified with keyword rules (passenger notices kept, corporate news skipped), dates are read from titles ('between 20.10.2025 and 24.10.2025', 'January 14, 15 and 16'), and stations are matched against CMRL's station list, which also gives each station's line (Blue / Green).

Robust parsing. Unclosed tags close implicitly when the next field starts; items without id, text or route/area are skipped with a diagnostic; duplicates and backwards windows are rejected. A page with no notices and no explicit 'empty' marker raises ParseError, so a layout change can never mass-withdraw data. One slow CMRL page (a real 25 s timeout on the first live run) now only skips that notice - its stored data stays, unverified - instead of failing the whole refresh.

Scheduling and alerts. A background thread wakes every 30 s and runs each source whose next_run_at is due. Changes are passed to the Alerter, which writes one alert_log row per (notice, revision, subscriber) with INSERT OR IGNORE, so repeated refreshes never resend. Demo mode writes to logs/alerts.log; SMTP is configured through environment variables.

Website. Every filter is a drop-down. Its options are computed from the notices that match all the other chosen filters, shown with counts, and zero-count options are hidden - so every choice returns results. Chosen filters appear as removable chips; notices in effect now are listed first; each card shows status in plain words, type, operator type, window, agency, 'checked N min ago' and the official link.

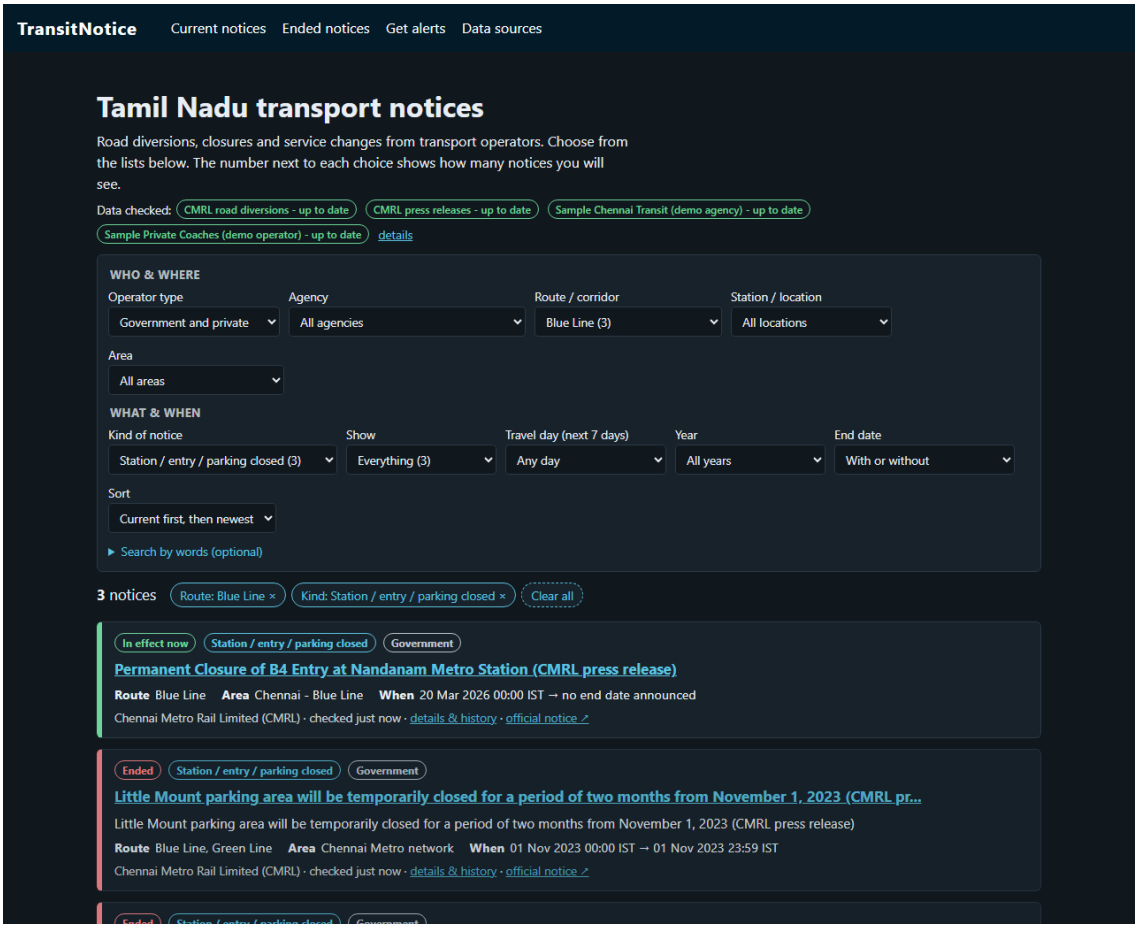


Figure 3 - Dashboard filtered to Blue Line + station / entry / parking closures: counts in every drop-down, removable chips, current notice first, ended notices clearly labelled.

5. Testing

59 automated tests (python -m unittest discover -s tests) - all pass. Fixtures cover malformed HTML, missing fields, duplicates, layout changes, unavailable sources, RSS/Atom feeds and CMRL page layouts.

Test (mandatory acceptance tests first)	Input	Expected	Result
Expired advisory not marked active	Window ended 1 Oct, now 2 Nov	Status 'Ended', only in archive	PASS
Route filter with multiple routes	Notice on Red + Orange	Found by 'Red' and by 'orange'	PASS
Extended advisory updates history	valid_to +1 day	Same notice, rev 2, 'Extended ...'	PASS
Time-zone handling reproducible	IST / EST / EDT / offsets / date-only	UTC values in policy table	PASS
Stale notice shows latest verification	Last success 5 h ago	'Stale - last verified 5 h ago'	PASS
Source error does not make old data live	HTTP 503 after a good run	last_confirmed unchanged; 'error'	PASS
Idempotent refresh / withdraw + reinstate	Same data twice; item removed then back	0 changes; withdrawn -> active rev 3	PASS
Malformed HTML, missing fields, duplicates	4 HTML fixtures	Good items kept, bad skipped + logged	PASS
Layout change / empty list	Error page; '[]'	ParseError, nothing withdrawn	PASS
Retry ceiling, backoff, robots.txt	503 forever; Disallow; robots 500	3 attempts (2 s, 4 s); denied	PASS
One alert per revision	3 refreshes + 1 revision, 2 subscribers	2 alerts, then 4	PASS
CMRL detail page down	1 of 3 pages returns 503	Run OK; that notice kept, not re-verified	PASS
Press-release classification and windows	12-row fixture	8 kept; exact IST windows; lines	PASS
Every drop-down option has results	All options, several filter combinations	Count shown = rows returned > 0	PASS
Security	No admin token; <script> in text	403; output escaped	PASS

Live checks on the running system (02 Nov 2026)

- CMRL diversions: 43 parsed, 0 skipped. CMRL press releases: 1,334 rows read, 261 passenger notices kept, 1 skipped (date written only as '2016').
- Mentor challenges on the demo source: extend SM-101 24 -> revision with 'Extended'; open-ended -> 'No end date announced'; break -> SOURCE ERROR, data kept, retry later; withdraw -> moved to archive.
- Filter crawl: all 129 options plus sampled pairs = 595 requests, **0 empty results**; dashboard with 312 notices renders in about 0.1 s.

6. Evaluation

Area	Result
Collection quality	100 % of diversions and 99.6 % of passenger press releases parsed. A keyword bug ('shuttle' matched 'shut') was found from screenshots and fixed with a test.
Alert precision	Alerts only when route, station, area and type filters all match; none for notices already ended when first seen; never repeated for the same revision.
Refresh performance	CMRL diversions: ~1 min on first run, then 2-3 requests; press releases: 2 requests (900 KB page); demo sources every 5 min.
Limitations	Press releases carry no validity: undated ones are current for 14 days; durations such as 'for two months' are not interpreted; PDF details are linked, not parsed; CMRL diversions have no end dates; SQLite suits a single instance.

7. Deployment and operations

- Install: python -m venv .venv, pip install -r requirements.txt (Flask, tzdata).
- Configure: copy .env.example to .env and set TN_ADMIN_TOKEN (long, random), TN_CONTACT, TN_ALERT_MODE (demo / smtp + SMTP settings). .env is git-ignored; agencies.json lists sources.
- Run: python run.py serve (web + scheduler); python run.py refresh for a one-off refresh; migrations apply automatically and only once.
- Monitor: GET /healthz for uptime; the Data sources page shows failures in a row, last error and next run; fetch_runs keeps every run with counts and diagnostics.

8. Ethics, compliance and conclusion

TransitNotice reads only public pages that robots.txt allows, identifies itself, rate-limits, prefers structured endpoints, and never logs in or bypasses protections. Sources whose policy could not be read were left out rather than guessed. Notices contain no personal data; subscribers give only an email and a filter. Every card shows freshness and links to the official notice, and the footer states that the site is unofficial. **Known gaps:** no unsubscribe link in alert emails yet, English only, no real private operator feed available. **Future work:** Tamil language interface, PDF text extraction for press releases, more agencies once their policies allow it, PostgreSQL, web-push alerts and a map view.

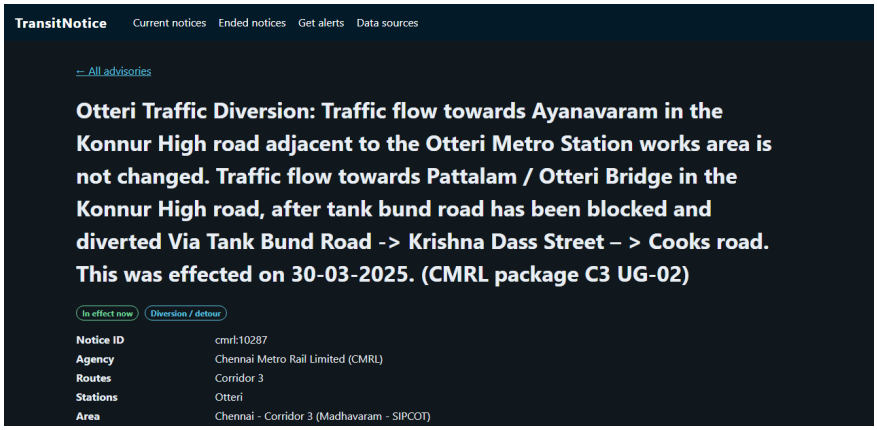
In conclusion, the project meets the brief: official Tamil Nadu notices are collected politely, normalised, versioned and archived; freshness and provenance are always visible; alerts are de-duplicated; failures degrade gracefully; and the website is usable without typing.

Appendix

A. Sample stored records (from /api/notices)

Field	Road diversion	Press release
notice_id	cmrl:10287	cmrl_press:Press-Release-21.10.2026-English.pdf
summary	Otteri Traffic Diversion: Traffic flow towards Ayanavaram ... is not changed.	Phase 2 Corridor-4 Stage 1A & 1B Passenger Service: Revised Service Timings
route_id / area	Corridor 3 / Chennai - Corridor 3 (Madhavaram - SIPCOT)	Corridor 4 / Chennai - Corridor 4
type / status	detour / active (no end date announced)	schedule_change / active
valid_from -> valid_to (UTC)	2025-03-29T18:30:00Z -> NULL	2026-10-20T18:30:00Z -> 2026-11-04T18:29:59Z
source_url	chennaimetrorail.org/road-diversion/otteri/	chennaimetrorail.org/.../Press-Release-21.10.2026-English.pdf

B. Notice detail page (official text and provenance; on the site the revision history is listed below)



All screenshots are in docs/img/: dashboard.png, filtered.png, notice.png, status.png, subscribe.png.

C. Git commit evidence: _____ (paste git log --oneline output here).